

Fast k NN Graph Construction with Locality Sensitive Hashing

Yan-Ming Zhang¹, Kaizhu Huang², Guanggang Geng³, and Cheng-Lin Liu¹

¹ National Laboratory of Pattern Recognition, Institute of Automation,
Chinese Academy of Sciences, Beijing, China
`{ymzhang, liuc1}@nlpr.ia.ac.cn`

² Department of EEE, Xi'an Jiaotong-Liverpool University, SuZhou, China
`kaizhu.huang@xjtlu.edu.cn`

³ China Internet Network Information Center,
Chinese Academy of Sciences, Beijing, China
`gengguanggang@cnnic.cn`

Abstract. The k nearest neighbors (k NN) graph, perhaps the most popular graph in machine learning, plays an essential role for graph-based learning methods. Despite its many elegant properties, the brute force k NN graph construction method has computational complexity of $O(n^2)$, which is prohibitive for large scale data sets. In this paper, based on the divide-and-conquer strategy, we propose an efficient algorithm for approximating k NN graphs, which has the time complexity of $O(l(d + \log n)n)$ only (d is the dimensionality and l is usually a small number). This is much faster than most existing fast methods. Specifically, we engage the locality sensitive hashing technique to divide items into small subsets with equal size, and then build one k NN graph on each subset using the brute force method. To enhance the approximation quality, we repeat this procedure for several times to generate multiple basic approximate graphs, and combine them to yield a high quality graph. Compared with existing methods, the proposed approach has features that are: (1) much more efficient in speed (2) applicable to generic similarity measures; (3) easy to parallelize. Finally, on three benchmark large-scale data sets, our method beats existing fast methods with obvious advantages.

Keywords: graph construction, locality sensitive hashing, graph-based machine learning.

1 Introduction

Graph-based learning methods present an important category of machine learning methods, and have been widely used in areas like image processing, computer vision, and data mining. These methods first represent the data set by a similarity graph, and then perform on this graph the traditional learning tasks, such as clustering [25], dimensionality reduction [1], and classification [34]. As observed by many researchers [28,33], the graph construction step plays an extremely important role to this kind of methods, and has attracted much attention recently.

Although various graph construction methods have been proposed to better describing the data set [19,9,7], the k NN graph still presents the most popular one in practice due to its robust performance. Given a data set, the k NN graph is constructed by connecting each item to k items which are the most similar to it under a given similarity measure. Despite its simplicity in concept, a direct implementation suffers from high computational cost. Since finding the k NN for each item needs $n - 1$ comparisons, it takes $O(n^2)$ time to construct the k NN graph. Obviously, it is too slow for large scale problems. In fact, after the recent development of fast graph-based learning methods [17,32], the speed of the graph construction method is becoming the bottleneck of graph-based methods.

To alleviate this problem, substantial efforts have been made to reduce the complexity of k NN graph construction. Early works [3,27] focused on constructing exact k NN graph. However, their complexity scales exponentially with data dimensionality. Recently, researchers switched their attention to construct approximate k NN graph and obtained encouraging results. These methods adopted techniques such as space partition tree [6,30], and local search [12]. A brief introduction to these methods can be seen in the next section.

Following the research direction, in this work, we propose a novel approximate k NN graph construction method which leads to significantly fast speed with high accuracy. Its basic idea is to divide the whole data set into small groups, and finds each item's k NN within the group it belongs to. Since the size of a group is much smaller than the size of the whole data set, the cost for finding the approximate k NN is much lower. To make this method work well, the key is to divide the data set in such a way that: (1) The pairwise similarity between items should be preserved such that similar items remain in the same group; (2) The group size should be kept as small as possible. We propose to group similar items by adopting locality sensitive hashing (LSH) [13], which enjoys a rigorous theoretical performance guarantee even in the worst case [18]. Further, we design a simple method to take control of the group size. As groups have no overlapping, the constructed graph is a union of multiple isolated small graphs. To improve the approximation quality, we repeat the division for several times to generate multiple basic approximate graphs. Finally, we combine them to yield a graph with high accuracy.

Compared with existing methods, we emphasize that the proposed method enjoys the following appealing advantages:

1. Fast and accurate. Our method has the time complexity of $O(l(d + \log n)n)$ (d is the dimensionality and l is usually a small number). This is much faster than most existing fast methods [12,6] (see the next section for details). Moreover, as shown by experiments, our method can generate good approximate k NN graphs by scanning a small proportion of pairwise distances. On several benchmarks, our method beats existing fast methods with obvious advantages.
2. Applicable to generic similarity measure. Thanks to the development of LSH, hash functions have been designed for different similarity measures, such as l_p [10], Mahalanobis distance [22], kernel similarity [21], and χ^2 distance [14].

Thus, we can conveniently use them to group data items according to the problem at hand; this presents one of the biggest advantages over the other methods such as [6] and [30], which can only be applied in Euclidean space.

3. Easy to parallelize. Since the construction of multiple basic approximate graphs are independent of each other, we can further speedup our method by constructing these basic graphs simultaneously.

The rest of this paper is organized as follows: the next section briefly reviews the existing approximation graph construction methods. Section 3 gives an introduction to the LSH technique, which provides the basic tools for our method. After that, section 4 presents a detailed description and complexity analysis of our method, while Section 5 gives comparison experimental results to validate the advantages of our method. Finally, we set out the conclusion in Section 7.

2 Related Work

The fast construction for k NN graph have been studied for a long time, and a comprehensive introduction to the graph construction methods can be found in [6]. In this section, we will briefly review several existing methods for approximate k NN graph construction, and a closely-related but different problem: the k NN search.

Chen, Fang, and Saad [6] proposed a divide-and-conquer style algorithm for constructing approximate k NN graph. The method recursively divides the data points into subsets **with overlapping**, then constructs one k NN graph on each small subset. The final graph is constructed by merging all the small graphs together using overlapping parts. Empirically, the authors reported their method had complexity of $O(dn^{1.22})$. Recently, Wang et al. [30] proposed another efficient algorithm using a similar idea. But data sets are recursively divided without overlapping. To increase the k NN recall, it constructs multiple basic graphs by repeating the division procedure for several times. To make good division, both methods use principle direction to partition data set. Thus, they need to compute $O(n)$ principle directions, one for each internal node. Although sub-sampling and Lanczos algorithm are adopted, it is still costly. Also, these methods can only be applied in Euclidean space.

Dong, Moses, and Li [12] proposed a fast k NN graph construction method based on local search. Its motivation lies in that **a neighbor of a neighbor is also likely to be a neighbor**. Initializing each node with a random set of neighbors, the method iteratively improves each node's neighborhood by exploring its neighbors' neighborhoods. Although the paper reported that its empirical cost was $O(n^{1.14})$, there is no formal guarantee on the algorithm complexity.

k NN search is a close-related but different problem which is extensively used in areas like information retrieval, pattern recognition. Given a query q , k NN search aims to find out the k most similar objects in the database. Thus, the construction of a k NN graph can be viewed as a k NN search problem where each data point itself is a query. Although many excellent works, such as space partition tree [2,26] and locality sensitive hashing (LSH) [13,10], have been done

for performing efficient k NN search, the direct use of k NN search approach for the graph construction results in unfavorable results [12,30]. The differences between the two problems lie in two aspects:

1. Since the prime concern of k NN search is to reduce the query time, these methods typically build elaborate indexing structure in the training step. However, there is no separated training phase for the graph construction problem, and we can not afford the high cost for building complex indexing structure.
2. k NN search is an inductive problem, which means we can not acquire query points in the training phase. However, k NN graph construction is a transductive problem in which all query points are at hand. Thus, k NN graph construction is easier in general, and we could take advantages of its characteristic to design more efficient algorithm.

Recently, Goyal, Daumé and Guerra [15] proposed an approximate graph construction method for natural language processing problem by adopting LSH technique. Basically, it applies the method of [5] which returns k approximate neighbors for each query in constant time by using two hash tables.

3 Locality Sensitive Hashing

Locality sensitive hashing (LSH) is an efficient technique for approximate k NN search problem. Because it serves as a foundation to the proposed method, we briefly introduce it as follows.

The core idea of LSH is to hash items in a similarity preserving way, i.e., it tries to store similar items in the same buckets, while keeping dissimilar items in different buckets. In general, LSH method for k NN search has two steps: training and querying. In the training step, LSH first learns a hash function $h(x) = \{h_1(x), h_2(x), \dots, h_m(x) : h_i(x) \in Z\}$ where m is the code length. For example, in the binary coding using linear projection, LSH adopts the hash function of form $h_i(x) = \text{sgn}(w_i^T x + b_i) \in \{-1, 1\}$, where $\{w_i, b_i\}_{i=1, \dots, m}$ are parameters to be learned. Then, LSH represents each item in the database as a hash code by the hash mapping $h(x)$, and constructs a hash table by hashing each item into the bucket indexed by its code. In the querying step, LSH first converts the query into a hash code, and then finds its approximate k NN in the bucket indexed by the code. One attractive feature of the LSH method is that it enjoys a rigorous theoretical performance guarantee even in the worst case [18]. In practice, it can provide constant or sub-linear search time.

Although the classic LSH method builds hashing codes by random projection, recent works focus on learning data-dependent hash functions so as to generate more accurate and compact hash codes to accelerate the query. As many machine learning techniques, such LSH methods can be divided into three main categories: supervised methods [24,4], semi-supervised methods [29] and unsupervised methods [31,23,20].

Most of LSH methods described before only work for l_2 similarity measure. To apply LSH for different similarity measure, researchers have designed different hash functions, such as l_p [10], Mahalanobis distance [22], kernel similarity [21], and χ^2 distance [14]. This provides another motivation of our method: according to the problem at hand, we can conveniently choose from these methods to group data items.

4 k NN Graph Construction with LSH

4.1 Problem Definition

Given a set of n items $S = \{x_1, x_2, \dots, x_n\}$ and a similarity measurement $\rho(x_i, x_j)$, the k NN graph for S is a directed graph that there is an edge from node i to j if and only if x_j is among x_i 's k most similar items in S under ρ . Here, ρ could be any similarity measurement defined on domain S . For example, it can be cosine similarity, kernel similarity, Mahalanobis distance etc.

4.2 Algorithm

The key idea of our method is to divide the whole data set into small groups, then find each item's k NN within the group it belongs to. Since it is very hard to ensure that each item and its real k NN are located in the same group, the method only provides an approximate result. From the perspective of graph construction, it is equivalent to say the method constructs one k NN graph on each group, and then takes the union of all these small k NN graphs as the approximation k NN graph.

Since the k NN search is performed within a small subset, finding one item's k NN needs only block-sz comparisons, where block-sz is the size of the group it locates in. Assuming all groups are of equal size, the method's complexity is $O(\text{block-sz} \times n)$ instead of $O(n^2)$ for the brute-force manner.

To make the strategy work well, two conditions should be satisfied:

1. Similar items should be grouped together, which implies most of the real k NN of an item can be found in its group. Therefore, one can expect the resulting graph is a good approximation to the true k NN graph.
2. Since the complexity of the method is $O(\text{block-sz} \times n)$, to make the method efficient, each group should be as small as possible ($\text{block-sz} \ll n$).

Obviously, there is a contradiction between the two conditions: to make Condition 1 valid, we tend to use groups of big size which will violate Condition 2. Thus, a balance should be made by choosing a feasible block-sz.

In this work, we explore LSH to divide the data set with the hope that similar items will be grouped together. A straightforward way is to use LSH to hash all items into a hash table, then construct a k NN graph for each bucket. However, typical LSH methods yield highly un-even hash table which means some buckets contain a large number of items and some contain few items as shown in Fig. 1.

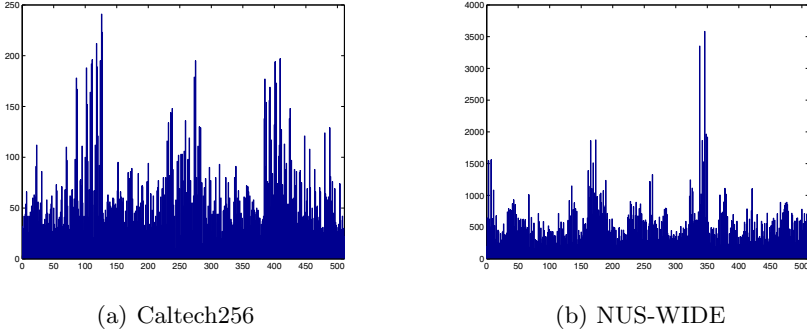


Fig. 1. Distribution of the number of items in buckets. Experiments are performed on Caltech256 and NUS-WIDE data sets. We use the function $h(x) = \{\text{sgn}(w_i^T x + b_i)\}_{i=1,\dots,8}$ to hash all items into a hash table, where $\{w_i\}$ are sampled from a Gaussian distribution $N(0, I)$ and $\{b_i\}$ are the median values of the projections. Since $m = 8$, the hash table has 512 buckets.

The problems brought by this phenomenon are: (a) the k NN graph constructed on a small bucket will fail the Condition 1, and suffer a poor precision; (b) Constructing a k NN graph on a large bucket will fail the Condition 2, and suffer a high cost because of the complexity $O(\text{block-sz}^2)$. Actually, the direct use of LSH has been observed to result in unsatisfying performance[12,30], which is also verified by our experiments.

We propose an efficient way to overcome the problem and obtain equal size groups. Given the data set's hash code matrix $Y \in \{0, 1\}^{n \times m}$ where the i th row $y_i \in \{0, 1\}^m$ is the hash code of x_i , we first project items' hash codes onto a random direction $w \in \mathbb{R}^m$, and get $p = Yw$. Then we sort items by their projection values to get the sequence $\{x_{\pi_1}, x_{\pi_2}, \dots, x_{\pi_n}\}$ with $\{p_{\pi_1} \leq p_{\pi_2} \leq \dots \leq p_{\pi_n}\}$. Finally, we obtain $n/\text{block-sz}$ groups $\{S_i\}$ with equal size of block-sz by defining $S_i = \{x_{\pi_{(i-1) \times \text{block-sz} + 1}}, \dots, x_{\pi_{i \times \text{block-sz}}}\}$. Because items in the same bucket will have same projection values, they will remain in the same group with high probability. We summarize the procedure described above in Algorithm 1.

Algorithm 1 generates a basic approximation to the k NN graph. However, it is just an union of $n/\text{block-sz}$ isolated small graphs, and may suffer from a low accuracy. To improve it, we repeat Algorithm 1 using different hash functions for multiple times and then combine the resulting graphs. Denoting the approximate k NN of x found in the i -th iteration by $N_i(x)$, we can obtain at most $k \times l$ NN different candidates as $\{N_i(x)\}_{i=1,\dots,l}$ after l iterations. Obviously, by increasing l , it will cover more and more true k NN of x . The final graph is achieved by connecting each x to k items in $\{N_i(x)\}_{i=1,\dots,l}$ which are the nearest to it.

To further boost our method, we use a one-step neighbor propagation procedure which is widely used in efficient k NN graph construction methods [6,12,30]. It is based on the following observation: if x is similar to y and y is similar to z , then it is likely that x is similar to z . It implies that we could improve an

```

Function: basic_ann_by_lsh(X, k, m, block-sz)
begin
   $Y = LSH(X, m)$ 
  Project  $Y$  onto a random direction  $w$ ,  $p = Yw$ 
  Sort items by  $p$  values, and get  $\{x_{\pi_1}, x_{\pi_2}, \dots, x_{\pi_n}\}$ 
  for  $i = 1, \dots, n/\text{block-sz}$  do
     $S_i = \{x_{\pi_{(i-1) \times \text{block-sz} + 1}}, \dots, x_{\pi_{i \times \text{block-sz}}}\}$   $g_i = \text{brute\_force\_kNN}(S_i, k)$ 
  return:  $G = \bigcup \{g_i\}$ 

```

Algorithm 1. Basic k NN Graph Construction with LSH

```

Algorithm: Approximate  $k$ NN Graph Construction with LSH
Input: X, k, l, m, block-sz
begin
  for  $i = 1, \dots, l$  do
     $G_i = \text{basic\_ann\_by\_lsh}(X, k, m, \text{block-sz})$ 
  Combine  $\{G_1, G_2, \dots, G_l\}$  to get  $G$ 
  Refine  $G$  by one step neighbor propagation
  return: G

```

Algorithm 2. Main Algorithm

item's approximate k NN by selecting from its neighbors and the neighbors of its neighbors. Formally, denoting the approximate k NN of x found by now as $N(x)$, we update it by reselecting x 's k NN from the set $N(x) \cup \{\cup_{v \in N(x)} N(v)\}$. As we will see in the next section, this simple local search procedure is a good complementary strategy with LSH, and gives significant improvement to the final results. The whole method is summarized in Algorithm 2.

In the proceeding of multiple basic graph constructions and neighbor propagation, some pairwise distances could be computed for many times. Thus, another technique to speedup the method is to store all the pairwise distances that have been computed so far in a hash table. When requiring the distance between two items, we first check the hash table if it has been evaluated before. However, we do not actually adopt this technique as the hash table may be too large to store. For example, assuming $n = 1,000,000$ and 10% of all the n^2 pairwise distances have been computed, the hash table needs about 8 GB memory to store these values.

4.3 Complexity Analysis

We briefly analyze the complexity of the proposed method as follows.

1. Complexity for computing $LSH(S, k)$, projecting and sorting are $O(nmd)$, $O(nm)$ and $O(n \log n)$ respectively. Complexity for the construction of all g_i ($1 \leq i \leq n/\text{block-sz}$) is $O(nd\text{block-sz})$. Thus, the complexity of Algorithm 1 is $O(n(md + m + d\text{block-sz}) + n \log n)$.

2. Combining l graphs needs $O(lnk)$ operations.
3. Each item has at most $k^2 + k$ candidate neighbors (k neighbors and k^2 neighbors of neighbor), and needs $O(dk^2)$ operations to find out the k nearest neighbors. Thus, the complexity of neighborhood propagation step is $O(ndk^2)$

Summarizing the above analysis, the complexity of our algorithm is $O(ln(md + d\text{block-sz} + \log n + k) + ndk^2)$. Since k , m and block-sz are small in practice, the actual complexity is $O(ln(d + \log n))$.

5 Experiments

To empirically validate the effectiveness of the proposed approximate k NN graph construction method, we provide results on several data sets. The primary goal is to verify: (1) The proposed method is much faster than existing ones; (2) Our method can generate good approximate graph by scanning only a small part of the total pairwise distances; (3) For many classification tasks, an approximate graph is enough to achieve good classification accuracy.

5.1 Experimental Setting

Data Set. We use 3 popular image data sets to evaluate the proposed method: Caltech256, Imagenet and NUS-WIDE. Caltech256 [16] is a benchmark for object classification, while Imagenet [11] and NUS-WIDE [8] are two real-world web image databases. Features are extracted by the bag-of-word (BOW) model based on SIFT descriptions. For Caltech256, we perform k-means clustering of SIFT descriptors to form a visual vocabulary of 1024 visual words. Then, SIFT descriptors are quantized into visual words using the nearest cluster center. BOW features for Imagenet and NUS-WIDE are directly downloaded from websites^{1,2}. For the Imagenet database, we only adopt the first 100 categories of the training set. We summarize the size and dimensionality of the data sets in Table 1.

Table 1. Data sets description

	Caltech256	Imagenet	NUS-WIDE
Size	30607	120955	269643
Dimensionality	1024	1000	500

Comparison Methods. We compare our method, denoted by LSH, with 3 popular approximate k NN graph construction methods: OverTree [6], MultiTree [30] and NN-Descent [12]. We employ the direct use of LSH to construct approximate

¹ <http://www.image-net.org/challenges/LSVRC/2010/download-public>

² <http://lms.comp.nus.edu.sg/research/NUS-WIDE.htm>

k NN graph as the baseline method ³, denoted by DirectLSH. To examine the effect of the one-step neighbor propagation, we will also report the results of our algorithm without the neighbor propagation procedure, denoted by LSH-. The setting for different methods is given as follows.

1. For LSH, LSH- and DirectLSH, we adopt a LSH coding method proposed in [23] to divide data points. It is a nonlinear method based on spectral decomposition of a low-rank graph matrix. To efficiently generate different hash tables, we randomly sample 20 items as the anchor points for each iteration. The length of the hash code is set to $m = \lceil \log_2(n/\text{block-sz}) \rceil + 1$. For LSH and LSH-, block-sz (the maximum set size for performing brutal force k NN) is set to 100.
2. For OverTree, the specific algorithm we use is k NN-glue, and the codes are provided by authors ⁴. block-sz is set to 100.
3. For MultiTree, we randomly sample 20 points on each internal node, compute the principal direction on the sampled set by Lanczos algorithm, then use it to perform the projection. block-sz is set to 100.
4. For NN-Descent, there is no hyperparameter to set, and the codes of NN-Descent are provided by authors ⁵.

Evaluation Criterion. To evaluate the quality of an approximate k NN graph G' , we define the following accuracy measurement $acc(G') = \frac{|E(G') \cap E(G)|}{|E(G)|}$, where G is the exact k NN graph, $E(\cdot)$ denotes the set of the directed edges in the graph and $|\cdot|$ denotes the cardinality of the set. The exact k NN graph is computed by the brute-force method, and the time for constructing 10NN graph is shown in Table 2.

Table 2. 10NN graph construction time for the brute-force method

	Caltech256	Imagenet	NUS-WIDE
Time (sec.)	1108	17991	43696

Experiment Environment. All experiments are run on a Linux server with two 8-core 2.66GHz CPUs and 128G RAM. Parallelization is disabled for all method, and each method is restricted on one thread.

5.2 Experimental Results

Time versus Approximate Accuracy. We test various methods by comparing their time for constructing graphs with different accuracies. To generate

³ In each iteration, we use LSH to group data, then construct one k NN graph for each bucket.

⁴ <http://www.mcs.anl.gov/~jiechen/software.html#knn>

⁵ <http://code.google.com/p/nndes/>

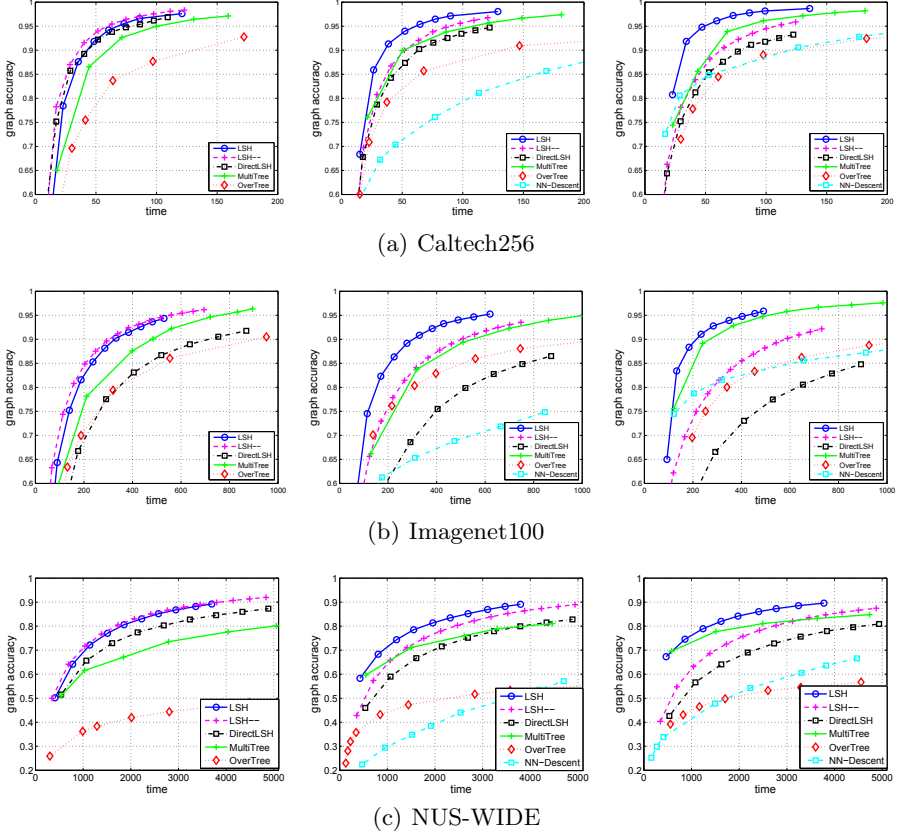


Fig. 2. Graph accuracy versus construction time for different methods on 3 data sets. The 3 columns correspond to the results of building approximate 1NN, 10NN and 20NN graphs respectively.

different accuracy graphs, (1) For LSH, LSH-, DirectLSH and MultiTree, we use different times of divisions (parameter l). (2) For OverTree, we vary the overlapping factor from $[0.1 \ 0.5]$. (3) For NN-Descend, we use different sample rates. The performance comparison is shown in Fig 2. The horizontal axis represents the time (in seconds) consumed for graph construction, and the vertical axis represents the graph accuracy defined above. Each row of the figure corresponds to one data set, and each column corresponds to a choice of k value. As the NN-Descend method fails for small k , its performance curve is not shown for $k = 1$.

From Fig. 2, we can clearly see the superiority of our method. For $k = 10$ and $k = 20$, LSH is consistently better than LSH-, and has similar performance for $k = 1$, which shows the effectiveness of the neighbor propagation procedure. On the other hand, the performance of LSH- is always better than DirectLSH,

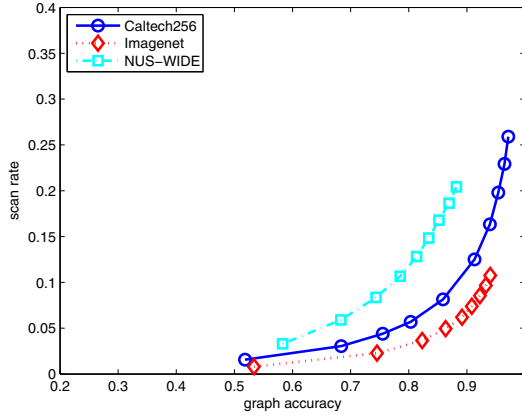


Fig. 3. Scan rate of the proposed method with respect to different graph accuracy on 3 data sets($k = 10$)

which shows the effectiveness of the equal size splitting. In fact, the accuracy of one basic isolated graph produced by DirectLSH is higher than that of LSH-, but it takes DirectLSH about 2 - 3 time than LSH-. Thus, in the same time our method builds more basic graphs than DirectLSH, and obtains higher accuracies.

Our method is at least 4 times faster than OverTree and NN-Descent methods on all data sets. MultiTree exhibits similar performance on Caltech256 and Imagenet, but is dramatically slower than the proposed method on NUS-WIDE. Furthermore, MultiTree and OverTree are only applicable for Euclidean space with l_2 norm. Compared with the brute force method, our approach generates graphs with accuracies of 90% using at most 5% time of the brute force method on Caltech256 and Imagenet, and using less than 10% time of the brute force method on NUS-WIDE.

Approximate Accuracy vs. Scan Rate. Because the dominant cost of k NN graph construction is the pairwise distance calculation, we evaluate our method by reporting the ratio of the number of actual distances calculated to the total number of pairwise distances $n(n-1)/2$. The values for graphs of different accuracies under different data sets are shown in Fig. 3. We can observe that: (1) Our method generates good approximate k NN graphs by scanning only a small proportion of the distances, but to generate graphs with high accuracy, the method still has to scan a large number of distances. This is because: neighbors of some items are far away, and locality-sensitive hashing in nature is not good at searching for such points. (2) Different data sets have very different properties. For example, constructing approximate k NN graphs for Imagenet is much easier than for NUS-WIDE.

In experiments, we found the scan rate of MultiTree is even lower than ours. However, because of computing principle directions, in constructing a basic approximate graph, it takes about 3 times of the running time of Algorithm 1.

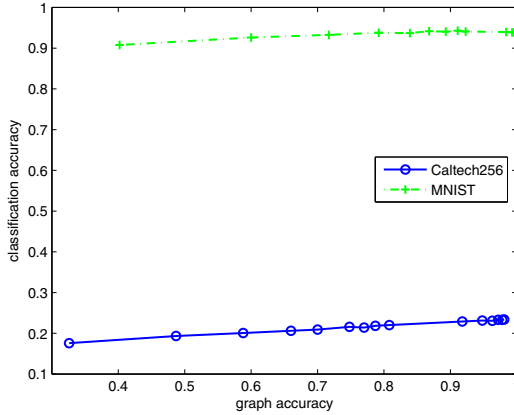


Fig. 4. Variance of classification accuracy with respect to graph accuracy

Thus, its overall speed is slow. Actually, by increasing the number of anchor points and using kmeans to choose anchor points (see experimental setting for LSH), our method can also achieve better scan rate. But this will make the cost of learning the hash coding much higher, and decrease the overall performance. Therefore, instead of pursuing the accurate but complex indexing structure as in k NN search, a trade-off must be made between the speed and accuracy in building the indexing structure for the graph construction problem.

Approximate Accuracy vs. Classification Accuracy. As we shown in the previous experiments, it is relatively easy to yield a good approximate graph, but hard to obtain graph with very high accuracy. The progress towards the real k NN graph becomes slow when l is large. Actually, this is a common problem shared by all approximate k NN graph construction methods. But fortunately for machine learning tasks, like classification, what we really care is not graph accuracy, but the classification accuracy. As long as edges found by our method are locally enough, we can expect the approximate graph leads to good description to the data set, and therefore yields similar results as the true k NN graph.

In this experiment, we examine the variance of classification accuracy with respect to the graph accuracy. First, approximate 20NN graphs with different accuracies are generated by our method on Caltech256 and MNIST(a popular handwritten digit data set with $n = 70000$, $d = 784$). We randomly sample 15420 nodes for Caltech256 and 700 nodes for MNIST as labeled data, and use Gaussian random fields [33] to classify the rest nodes. The classification accuracies are reported in Fig. 4.

As observed from the results, the classification accuracy is very robust with the graph accuracy. Actually, on Caltech256, the classification accuracy is 22.0% on graph of accuracy 80%, while is 23.4% on exact 20NN graph; on MNIST, the classification accuracy is 93.8% on graph of accuracy 80%, while is 93.9% on exact 20NN graph. This implies that for many applications, it is sufficient to construct a graph with a reasonable accuracy.

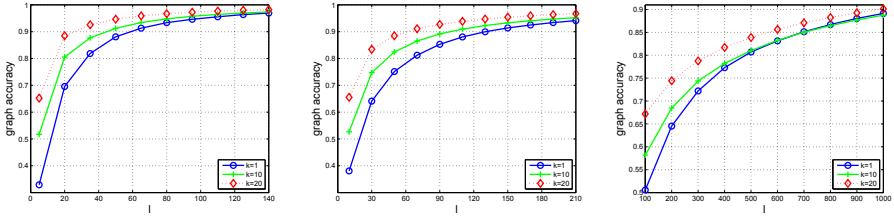


Fig. 5. Variance of classification accuracy with respect to the parameter l for building approximate 10NN graphs on Caltech256, Imagenet and NUS-WIDE respectively

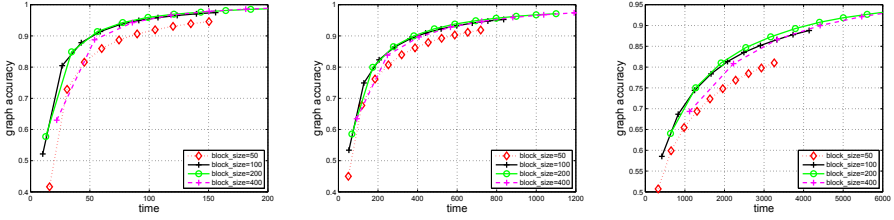


Fig. 6. Effect of the size of block for building approximate 10NN graphs on Caltech256, Imagenet and NUS-WIDE respectively

Approximate Accuracy vs. The Number of Divisions. In the proposed method, we use the number of divisions l to control the speed and accuracy of graphs. In this experiment, we examine how the graph accuracy varies with the number of divisions. For each data set, we construct $k = \{1, 10, 20\}$ NN graphs with different l , and report the accuracy of resulting graphs. The results are shown in Fig. 5.

Effect of the Size of Block. From the analysis in Section 4.2, we should choose a feasible block size to make a balance between the accuracy and the cost of basic graph. In this experiment, we examine how the block size affects the performance of our method. We construct approximate 10NN graphs on Caltech256, Imagenet and NUS-WIDE data sets with $\text{block_sz} \in \{50, 100, 200, 400\}$. For each data set, we build graphs with different accuracies by using different number of divisions, and report the building time and accuracies in Fig 6. As we can see, $\text{block_sz} \in \{100, 200\}$ are consistently better than $\{50, 400\}$ for all data sets. This is because $\text{block_sz} = 50$ is too small to generate a good basic graph while the cost for building basic graph with $\text{block_sz} = 400$ is too high. Actually, for all the previous experiments, we have fixed block_sz to 100.

6 Conclusion

In this paper, we have proposed a fast approximate k NN graph construction method for generic similarity measures. It engages the LSH technique to

efficiently partition items into small groups, and then constructs k NN graph on each group. As shown by the experiments, our method can efficiently generate graphs with good accuracies by just computing a small proportion of distances, which is sufficient for many learning tasks. In addition to its high efficiency, the proposed algorithm is ready to be applied under many similarity measures, and is also natural to be parallelized.

Acknowledgements. This work has been supported in part by the National Basic Research Program of China (973 Program) Grant 2012CB316301, the National Natural Science Foundation of China (NSFC) Grants 61203296, 61075052 and 61005029, Tsinghua National Laboratory for Information Science and Technology (TNList) Cross-discipline Foundation.

References

1. Belkin, M., Niyogi, P.: Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation* 15(6), 1373–1396 (2003)
2. Bentley, J.L.: Multidimensional binary search trees used for associative searching. *Communications of the ACM* 18(9), 509–517 (1975)
3. Bentley, J.L.: Multidimensional divide-and-conquer. *Communications of the ACM* 23(4), 214–229 (1980)
4. Bronstein, M.M., Fua, P.: LDAHash: Improved matching with smaller descriptors. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 34(1), 66–78 (2012)
5. Charikar, M.S.: Similarity estimation techniques from rounding algorithms. In: *Proceedings of the Thirty-Fourth Annual ACM Symposium on Theory of Computing* (2002)
6. Chen, J., Fang, H., Saad, Y.: Fast approximate k NN graph construction for high dimensional data via recursive lanczos bisection. *The Journal of Machine Learning Research* 10, 1989–2012 (2009)
7. Cheng, B., Yang, J.C., Yan, S.C., Fu, Y., Huang, T.: Learning with l1-graph for image analysis. *IEEE Transaction on Image Processing* 19, 858–866 (2010)
8. Chua, T.-S., Tang, J., Hong, R., Li, H., Luo, Z., Zheng, Y.-T.: NUS-WIDE: A real-world web image database from national university of singapore. In: *Proceedings of ACM Conference on Image and Video Retrieval* (2009)
9. Daitch, S.I., Kelner, J.A., Spielman, D.A.: Fitting a graph to vector data. In: *Proceedings of the International Conference on Machine Learning* (2009)
10. Datar, M., Immorlica, N., Indyk, P., Mirrokni, V.S.: Locality-sensitive hashing scheme based on p -stable distributions. In: *Proceedings of the Annual Symposium on Computational Geometry* (2004)
11. Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., Fei-Fei, L.: ImageNet: A Large-Scale Hierarchical Image Database. In: *IEEE Conference on Computer Vision and Pattern Recognition* (2009)
12. Dong, W., Charikar, M., Li, K.: Efficient k -nearest neighbor graph construction for generic similarity measures. In: *Proceedings of the International Conference on World Wide Web* (2011)
13. Gionis, A., Indyk, P., Motwani, R.: Similarity search in high dimensions via hashing. In: *Proceedings of the International Conference on Very Large Data Bases* (1999)

14. Gorisse, D., Cord, M., Precioso, F.: Locality-sensitive hashing for chi2 distance. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 34(2), 402–409 (2012)
15. Goyal, A., Daumé III, H., Guerra, R.: Fast large-scale approximate graph construction for nlp. In: *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning* (2012)
16. Griffin, G., Holub, A., Perona, P.: Caltech-256 object category dataset. Technical report, California Institute of Technology (2007)
17. Herbster, M., Pontil, M., Galeano, S.R.: Fast predciton on a tree. In: *Advances in Neural Information Processing Systems* (2008)
18. Indyk, P., Motwani, R.: Approximate nearest neighbors: towards removing the curse of dimensionality. In: *Proceedings of the Annual ACM Symposium on Theory of Computing* (1998)
19. Jebara, T., Wang, J., Chang, S.F.: Graph construction and b-matching for semi-supervised learning. In: *Proceedings of the International Conference on Machine Learning* (2009)
20. Kong, W., Li, W.J.: Isotropic hashing. In: *Advances in Neural Information Processing Systems* (2012)
21. Kulis, B., Grauman, K.: Kernelized locality-sensitive hashing for scalable image search. In: *IEEE International Conference on Computer Vision* (2009)
22. Kulis, B., Jain, P., Grauman, K.: Fast similarity search for learned metrics. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 31(12), 2143–2157 (2009)
23. Liu, W., Wang, J., Kumar, S., Chang, S.F.: Hashing with graphs. In: *Proceedings of the International Conference on Machine Learning* (2011)
24. Salakhutdinov, R., Hinton, G.: Semantic hashing. *International Journal of Approximate Reasoning* 50(7), 969–978 (2009)
25. Shi, J., Malik, J.: Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22(8), 888–905 (2002)
26. Uhlmann, J.K.: Satisfying general proximity/similarity queries with metric trees. *Information Processing Letters* 40(4), 175–179 (1991)
27. Vaidya, P.M.: An $O(n \log n)$ algorithm for the all-nearest-neighbors problem. *Discrete & Computational Geometry* 4(1), 101–115 (1989)
28. Von Luxburg, U.: A tutorial on spectral clustering. *Statistics and Computing* 17(4), 395–416 (2007)
29. Wang, J., Kumar, S., Chang, S.F.: Semi-supervised hashing for scalable image retrieval. In: *IEEE Conference on Computer Vision and Pattern Recognition* (2010)
30. Wang, J., Wang, J., Zeng, G., Tu, Z., Gan, R., Li, S.: Scalable k-NN graph construction for visual descriptors. In: *IEEE Conference on Computer Vision and Pattern Recognition* (2012)
31. Weiss, Y., Torralba, A., Fergus, R.: Spectral hashing. In: *Advances in Neural Information Processing Systems* (2008)
32. Zhang, Y.M., Huang, K., Liu, C.L.: Fast and robust graph-based transductive learning via minimum tree cut. In: *IEEE International Conference on Data Mining* (2011)
33. Zhu, X.: Semi-supervised learning literature survey. Technical report, Computer Science, University of Wisconsin-Madison (2008)
34. Zhu, X., Ghahramani, Z., Lafferty, J.: Semi-supervised learning using Gaussian fields and harmonic functions. In: *Proceedings of the International Conference on Machine Learning* (2003)